

Introducing asynchronous JavaScript

Overview: [Asynchronous JavaScript](#)

Next →

In this article, we'll explain what asynchronous programming is, why we need it, and briefly discuss some of the ways asynchronous functions have historically been implemented in JavaScript.

Prerequisites:	A solid understanding of JavaScript fundamentals .
Learning outcomes:	- To gain familiarity with what asynchronous JavaScript is, how it differs from synchronous JavaScript, and why we need it. - What synchronous programming is, and why it can sometimes be problematic. - How asynchronous programming aims to solve these problems. - Event handlers and callback functions, and how they relate to asynchronous programming.

Asynchronous programming is a technique that enables your program to start a potentially long-running task and still be able to be responsive to other events while that task runs, rather than having to wait until that task has finished. Once that task has finished, your program is presented with the result.

Many functions provided by browsers, especially the most interesting ones, can potentially take a long time, and therefore, are asynchronous. For example:

- Making HTTP requests using `fetch()`
- Accessing a user's camera or microphone using `getUserMedia()`
- Asking a user to select files using `showOpenFilePicker()`

So even though you may not have to *implement* your own asynchronous functions very often, you are very likely to need to *use* them correctly.

In this article, we'll start by looking at the problem with long-running synchronous functions, which make asynchronous programming a necessity.

Synchronous programming

Consider the following code:

JS

```
const name = "Miriam";
const greeting = `Hello, my name is ${name}!`;
console.log(greeting);
// "Hello, my name is Miriam!"
```

This code:

1. Declares a string called `name`.

2. Declares another string called `greeting`, which uses `name`.
3. Outputs the greeting to the JavaScript console.

We should note here that the browser effectively steps through the program one line at a time, in the order we wrote it. At each point, the browser waits for the line to finish its work before going on to the next line. It has to do this because each line depends on the work done in the preceding lines.

That makes this a **synchronous program**. It would still be synchronous even if we called a separate function, like this:

JS

```
function makeGreeting(name) {  
  return `Hello, my name is ${name}!`;  
}  
  
const name = "Miriam";  
const greeting = makeGreeting(name);  
console.log(greeting);  
// "Hello, my name is Miriam!"
```

Here, `makeGreeting()` is a **synchronous function** because the caller has to wait for the function to finish its work and return a value before the caller can continue.

A long-running synchronous function

What if the synchronous function takes a long time?

The program below uses a very inefficient algorithm to generate multiple large prime numbers when a user clicks the "Generate primes" button. The higher the number of primes a user specifies, the longer the operation will take.

HTML

```
<label for="quota">Number of primes:</label>  
<input type="text" id="quota" name="quota" value="1000000" />  
  
<button id="generate">Generate primes</button>  
<button id="reload">Reload</button>  
  
<div id="output"></div>
```

JS

```
const MAX_PRIME = 1000000;  
  
function isPrime(n) {  
  for (let i = 2; i <= Math.sqrt(n); i++) {  
    if (n % i === 0) {  
      return false;  
    }  
  }  
}
```

```

    }
    return n > 1;
}

const random = (max) => Math.floor(Math.random() * max);

function generatePrimes(quota) {
    const primes = [];
    while (primes.length < quota) {
        const candidate = random(MAX_PRIME);
        if (isPrime(candidate)) {
            primes.push(candidate);
        }
    }
    return primes;
}

const quota = document.querySelector("#quota");
const output = document.querySelector("#output");

document.querySelector("#generate").addEventListener("click", () => {
    const primes = generatePrimes(quota.value);
    output.textContent = `Finished generating ${quota.value} primes!`;
});

document.querySelector("#reload").addEventListener("click", () => {
    document.location.reload();
});

```

Number of primes:

Try clicking "Generate primes". Depending on how fast your computer is, it will probably take a few seconds before the program displays the "Finished!" message.

The trouble with long-running synchronous functions

The next example is just like the last one, except we added a text box for you to type in. This time, click "Generate primes", and try typing in the text box immediately after.

You'll find that while our `generatePrimes()` function is running, our program is completely unresponsive: you can't type anything, click anything, or do anything else.

Number of primes: Generate primes Reload

Try typing in here immediately after pressing "Generate primes"

The reason for this is that this JavaScript program is *single-threaded*. A thread is a sequence of instructions that a program follows. Because the program consists of a single thread, it can only do one thing at a time: so if it is waiting for our long-running synchronous call to return, it can't do anything else.

What we need is a way for our program to:

1. Start a long-running operation by calling a function.
2. Have that function start the operation and return immediately, so that our program can still be responsive to other events.
3. Have the function execute the operation in a way that does not block the main thread, for example by starting a new thread.
4. Notify us with the result of the operation when it eventually completes.

That's precisely what asynchronous functions enable us to do. The rest of this module explains how they are implemented in JavaScript.

Event handlers

The description we just saw of asynchronous functions might remind you of event handlers, and if it does, you'd be right. Event handlers are really a form of asynchronous programming: you provide a function (the event handler) that will be called, not right away, but whenever the event happens. If "the event" is "the asynchronous operation has completed", then that event could be used to notify the caller about the result of an asynchronous function call.

Some early asynchronous APIs used events in just this way. The `XMLHttpRequest` API enables you to make HTTP requests to a remote server using JavaScript. Since this can take a long time, it's an asynchronous API, and you get notified about the progress and eventual completion of a request by attaching event listeners to the `XMLHttpRequest` object.

The following example shows this in action. Press "Click to start request" to send a request. We create a new `XMLHttpRequest` and listen for its `loadend` event. The handler logs a "Finished!" message along with the status code.

After adding the event listener we send the request. Note that after this, we can log "Started XHR request": that is, our program can continue to run while the request is going on, and our event handler will be called when the request is complete.

HTML

```
<button id="xhr">Click to start request</button>
<button id="reload">Reload</button>

<pre class="event-log"></pre>
```

JS

```

const log = document.querySelector(".event-log");

document.querySelector("#xhr").addEventListener("click", () => {
  log.textContent = "";

  const xhr = new XMLHttpRequest();

  xhr.addEventListener("loadend", () => {
    log.textContent = `${log.textContent}Finished with status: ${xhr.status}`;
  });

  xhr.open(
    "GET",
    "https://raw.githubusercontent.com/mdn/content/main/files/en-us/_wikihistory.json",
  );
  xhr.send();
  log.textContent = `${log.textContent}Started XHR request\n`;
});

document.querySelector("#reload").addEventListener("click", () => {
  log.textContent = "";
  document.location.reload();
});

```

Click to start request

Reload

This is an [event handler](#) just the same as handlers for user actions such as the user clicking a button. This time, however, the event is a change in the state of an object.

Callbacks

An event handler is a particular type of callback. A callback is just a function that's passed into another function, with the expectation that the callback will be called at the appropriate time. As we just saw, callbacks used to be the main way asynchronous functions were implemented in JavaScript.

However, callback-based code can get hard to understand when the callback itself has to call functions that accept a callback. This is a common situation if you need to perform some operation that breaks down into a series of asynchronous functions. For example, consider the following:

JS

```

function doStep1(init) {
  return init + 1;
}

```

```
function doStep2(init) {
  return init + 2;
}

function doStep3(init) {
  return init + 3;
}

function doOperation() {
  let result = 0;
  result = doStep1(result);
  result = doStep2(result);
  result = doStep3(result);
  console.log(`result: ${result}`);
}

doOperation();
```

Here we have a single operation that's split into three steps, where each step depends on the last step. In our example, the first step adds 1 to the input, the second adds 2, and the third adds 3. Starting with an input of 0, the end result is 6 ($0 + 1 + 2 + 3$). As a synchronous program, this is very straightforward. But what if we implemented the steps using callbacks?

JS

```
function doStep1(init, callback) {
  const result = init + 1;
  callback(result);
}

function doStep2(init, callback) {
  const result = init + 2;
  callback(result);
}

function doStep3(init, callback) {
  const result = init + 3;
  callback(result);
}

function doOperation() {
  doStep1(0, (result1) => {
    doStep2(result1, (result2) => {
      doStep3(result2, (result3) => {
        console.log(`result: ${result3}`);
      });
    });
  });
}

doOperation();
```

Because we have to call callbacks inside callbacks, we get a deeply nested `doOperation()` function, which is much harder to read and debug. This is sometimes called "callback hell" or the "pyramid of doom" (because the indentation looks like a pyramid on its side).

When we nest callbacks like this, it can also get very hard to handle errors: often you have to handle errors at each level of the "pyramid", instead of having error handling only once at the top level.

For these reasons, most modern asynchronous APIs don't use callbacks. Instead, the foundation of asynchronous programming in JavaScript is the `Promise`, and that's the subject of the next article.

[Overview: Asynchronous JavaScript](#)

[Next →](#)



Your blueprint for a better internet.